Student Enrollment & Course Management System

JDBC Console Application — Project Assessment Brief

Objective

Build a console-based Java application using JDBC to manage student records and their course enrollments, persisting data in a relational database (MySQL/PostgreSQL), following layered architecture (DTO, Repository, Service, Util, Main).

Tech Stack

- Java (Core + JDBC)
- MySQL / PostgreSQL
- Maven or plain JDK (specify one for grading consistency)
- No frameworks (no Spring/Hibernate) — pure JDBC, to test fundamentals

Database Schema (to be created by student)

Table: **student**

Column	Type
id	INT PRIMARY KEY
first_name	VARCHAR(50)
last_name	VARCHAR(50)
gender	VARCHAR(10)
city	VARCHAR(50)
state	VARCHAR(50)
mobile_number	VARCHAR(15)
email_id	VARCHAR(100) UNIQUE
course_id	INT
course_name	VARCHAR(50)
enrollment_date	DATE
course_type	VARCHAR(20)
grade	VARCHAR(5)

Package Structure (mandatory)

```
com.jdbc.dto -> StudentDTO
com.jdbc.enums -> Gender, CourseType, Grade
com.jdbc.exception -> StudentNotFoundException, DuplicateStudentException
com.jdbc.connection -> DBConnectionUtil (singleton JDBC connection)
com.jdbc.repository -> StudentRepository (interface)
com.jdbc.repository.impl -> StudentRepositoryImpl (JDBC implementation)
```

```
com.jdbc.service -> StudentService (interface)
com.jdbc.service.impl -> StudentServiceImpl
com.jdbc.util -> InputUtil
com.jdbc.main -> App (main class)
```

Functional Requirements (Menu-driven)

1. Add new student with course enrollment details
2. Fetch student details by student id
3. Update course details of a student
4. Delete a student record
5. Fetch all students
6. Fetch student by email id and enrollment date
7. Exit

Technical Requirements

• Use **PreparedStatement** only (no raw Statement) to prevent SQL injection

• Use **try-with-resources** for Connection, PreparedStatement, ResultSet

• Implement a **DBConnectionUtil** class to centralize connection creation (read DB URL/username/password from a config file or constants)

• Handle **SQLException** explicitly and convert/wrap into meaningful custom exceptions where applicable

• Use **enums** for gender, course type, and grade — validate user input against enum values

• Use **LocalDate** for enrollment date with a fixed format (dd-MM-yyyy) for input parsing

• Throw and handle **StudentNotFoundException** when an id/email+date lookup fails

• Throw and handle **DuplicateStudentException** when inserting a student with an existing id

• All console I/O must be isolated in **InputUtil** (no Scanner calls inside Service/Repository)

• Main class (**App**) should only orchestrate: read input → call service → print output, no business logic in main

Evaluation Rubric

Criteria	Marks
Correct package/layered architecture (DTO/Repo/Service/Util/Main separation)	15
JDBC connection setup & resource management (try-with-resources, no leaks)	15
PreparedStatement usage (all 6 operations correctly implemented)	20
Exception handling (custom exceptions + SQLException handling)	15
Enum usage & validation	10
Input validation & recursive retry on bad input	10
Code quality (naming, comments, no business logic leaking into wrong layer)	10
Working demo / all menu options functional	5
Total	**100**

Submission Requirements

- Source code as a zip or GitHub repo link
- SQL script to create the student table
- A README with steps to run the project (DB setup, config, how to run)
- Screenshots or sample console output for all 6 operations

Suggested Time Frame

3–5 days, depending on whether students are already familiar with JDBC basics.